20. Софтуерна архитектура: Проектиране и документиране

1. Понятие за софтуерна архитектура

Дефиниция и същност

Софтуерната архитектура на една система представлява съвкупност от структури, които показват различните софтуерни елементи, техните външно видими свойства и връзките помежду им. Тя е **абстракция**, която разглежда елементите като „черни кутии“, фокусирайки се върху техните взаимодействия, а не върху вътрешната им реализация (алгоритми и детайли).

Структури и изгледи (Structures and Views)

Важно е да се прави разлика между структура и изглед:

- **Структура:** Реалната съвкупност от софтуерни елементи и техните връзки.
- **Изглед (View):** Конкретно документирано представяне на дадена структура, предназначено за комуникация със заинтересованите лица.

Основни групи архитектурни структури:

1. **Модулни структури:** Описват функционалните компоненти (например модули за продукти, плащания, потребители).
2. **Структури на процесите:** Описват динамичното поведение и комуникацията по време на изпълнение (например как заявка преминава през слоевете).
3. **Структури на разположението (Физически):** Показват разпределението на софтуерните компоненти върху хардуерни ресурси (сървъри, бази данни, кеш сървъри).

Основни архитектурни изгледи:

- **Логически изглед:** Абстракции като обекти и класове.
- **Изглед на процесите:** Взаимодействащи си процеси в реално време.
- **Изглед на кода:** Организация на файловете и пакетите.
- **Физически изглед:** Разположение върху хардуерни възли.

Необходимост и влияние

Архитектурата е необходима за управление на сложността в големи системи. Тя осигурява обща картина и език за комуникация между разработчици, тестери и мениджъри.

- **Влияние върху проекта:** Определя решения, които са трудни за промяна и насочват всички последващи действия.
- **Влияние върху организацията:** Структурата на екипите често следва архитектурата на системата (например отделни екипи за различни услуги или слоеве).

2. Архитектурни стилове

Архитектурните стилове са общи шаблони за организация, които влияят пряко върху качествените характеристики на системата.

Стил	Описание	Предимства/Недостатъци
Многослоен (Layered)	Вертикално разделени слоеве. Всеки слой предлага услуги на горния и ползва долния.	Ясно разделение на отговорностите; риск от намалена производителност.
MVC (Model-View-Controller)	Разделя данните (Модел), представянето (Изглед) и логиката на управление (Контролер).	Позволява една логика да има различни интерфейси; улеснява тестването.
Поточен (Pipe-and-filter)	Последователност от филтри, свързани чрез канали (pipes). Всеки филтър трансформира данни.	Възможност за паралелизъм и повторна употреба; разходи за парсване на данни.
Ориентиран към услуги (SOA)	Система от самостоятелни услуги със строго дефинирани интерфейси (договори).	Гъвкавост и слаба свързаност; сложност при мрежова комуникация.
Клиент-сървър	Сървър предлага ресурси, а клиентът ги консумира.	Класическо разпределение на отговорностите.
Shared-data	Данните служат като конектор. Вариации: Repository и Blackboard.	Централизирано управление на състоянието.

3. Проектиране на софтуерната архитектура

Процес на проектиране

Проектирането е **цикличен процес**, който започва с анализ на нуждите на заинтересованите лица (stakeholders).

1. **Идентифициране на драйвери:** Избор на най-приоритетните функционални изисквания (use-cases) и сценарии за качество.
2. **Декомпозиция:** Разделяне на системата йерархично на модули.
3. **Избор на тактики и шаблони:** Прилагане на конкретни решения за постигане на драйверите.
4. **Валидация:** Проверка дали декомпозицията отговаря на изискванията.

Сценарии за качество

Качеството се формализира чрез сценарии, състоящи се от 6 компонента:

- **Контекст:** Обстоятелства (например "по време на работа").
- **Източник:** Кой инициира събитието (например "потребител").
- **Въздействие (Стимул):** Какво се случва (например "прави справка").
- **Обект:** Коя част от системата е засегната.
- **Резултат:** Как реагира системата.
- **Количествени параметри:** Измерими критерии (например "за по-малко от 15 сек").

Тактики за качествени показатели

Тактиките са конкретни технически решения:

- **Надеждност:** Ping/echo, Heartbeat, активен/пасивен излишък (redundancy).
- **Производителност:** Кеширане, Load-balancing (разпределение на натоварването), паралелна обработка.
- **Сигурност:** Криптиране, хеширане на пароли, двуфакторна автентикация (2FA).
- **Скалируемост:** Хоризонтално (scaling out) и вертикално (scaling up) скалиране, sharding на бази данни.
- **Изменяемост:** Прилагане на принципа Open-Closed.

4. Документиране на софтуерната архитектура

Предназначение и принципи

Документацията служи за комуникация, поддръжка, обучение на нови кадри и основа за разработка.

- **Основен принцип:** Документацията трябва да е съобразена с нейните читатели. Тя трябва да е точна, актуална и да обяснява не само „какво“, но и „защо“ е взето дадено решение.

Съдържание и структура на документацията

Един стандартен архитектурен документ включва:

1. **Въведение:** Цели и обхват.
2. **Архитектурни изгледи:** Детайлни описания (логически, физически и др.).
3. **Сценарии за качество:** Описание на ключовите нефункционални изисквания.
4. **Архитектурни решения:** Обосновка на избраните стилове и тактики (включително разгледани алтернативи).
5. **Елементи и връзки:** Детайлно описание на интерфейсите.
6. **Терминологичен речник:** Дефиниции за избягване на двусмислие.